

Lab1:DataLab 实验报告

梁心一 25800190015

一、要求截图

1. 符号检查截图:

```
jimmy@Lxy-MeCr:~/ICS2026/Lab1DataLab$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 12/18 operators
secondLowestZeroBit: 6/8 operators
oddParity: 12/56 operators
rotateRightBits: 2/16 operators
roundEvenPow2: 11/24 operators
midpointTowardFirst: 18/32 operators
isBetweenEitherOrder: 29/48 operators
mul5Sat: 21/30 operators
classifyAdd3: 31/52 operators
floatScaleThreeHalves: 36/60 operators
floatRoundEven: 26/65 operators
float_i2f: 31/40 operators
bitCount: 36/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

2. 测试截图

```
jimmy@Lxy-MeCr:~/ICS2026/Lab1DataLab$ ./btest
Score Rating Errors Function
1      1      0    signMask
2      2      0    bitXor
3      3      0    negativePart
4      4      0    copyByteWithin
4      4      0    logicalShift
4      4      0    swapNibblePairs
4      4      0    secondLowestZeroBit
5      5      0    oddParity
5      5      0    rotateRightBits
5      5      0    roundEvenPow2
5      5      0    midpointTowardFirst
7      7      0    isBetweenEitherOrder
7      7      0    mul5Sat
7      7      0    classifyAdd3
7      7      0    floatScaleThreeHalves
10     10     0    floatRoundEven
10     10     0    float_i2f
10     10     0    bitCount
10     10     0    bitReverse
Total points: 110/110
```

二、程序思路（与代码注释相同）

P1 signMask:

1<<31 将1左移31位，得到 0x80000000。

P2 bitXor:

$\sim(x \& y)$ 生成剔除 x 和 y 中都为 1 的位的掩码

$\sim(\sim x \& \sim y)$ 生成 $x|y$ ，与掩码按位与就得到异或

P3 negativePart:

y 是 x 的负值， $x \gg 31$ ，由于算数右移，根据 x 的正负生成全 0 或者全 1 的掩码

P4 copyByteWithin:

1. 存下 src, dst 表示的位移数量，节省<<的出现次数

2. 用 byte 提取 src 位置的字节
3. 构造在 dst 位置位 0 的掩码, 将 dst 位置的字节清 0
4. 将提取的字节写入 dst 位置

P5 logicalShift:

构造最高 n 位全是 0, 其余是 1 的掩码, 将其与算术右移的结果 &

P6 swapNibblePairs:

构造 x 左移四位和右移四位, & 两个掩码得到交换结果的一半, 再相加得到结果

P7 secondLowestZeroBit:

用 $x | (x+1)$ 去掉最低位的 0, 取反后用 lowbit 函数找最小的 1 (即原本次小的 0), 如果没找到说明原本 0 不够 2 个, 自然返回 0

P8 oddParity:

二分法, 每次把前一半的和后一半的异或, 最后最低位就是全部 32 位异或的结果

P9 rotateRightBits:

1. 将 n 取模, 防止溢出
2. 标准逻辑右移得到 rx
3. 将 x 先左移 31-n 位再左移 1 位, 利用左移自动溢出抹去高位的特性获取循环回高位的左半部分 lx。
4. 按位或拼接 rx 与 lx 得到最终结果。

P10 roundEvenPow2:

1. h 代表半数 $1 \ll (n-1)$, b 为基础偏置量 h-1。
2. 利用 $(x+b+((x>n)&1))>>n$ 计算商: 大于 h 自动进位, 等于 h 则仅在商为奇数时进位, 小于 h 不进位。
3. 将计算得到的商左移 n 位恢复。

P11 midpointTowardFirst:

1. 利用 $(x&y)+((x^y)>>1)$ 计算无溢出基础平均值。
2. 当 x+y 为奇数 (diff&1 为 1) 时, 右移会自动向下取整偏向较小值:
若 $x>y$, 结果偏向于较小者 y, 需+1 向较大者 x 靠拢;
若 $x<y$, 结果已天然偏向较小者 x, 无需修正。
3. 分符号无溢出判断 $x>y$:
异号时: x 非负即大于 y, 取 $(\sim x>>31)&1$;
同号时: y-x 为负即 $x>y$, 取 $((y+\sim x)>>31)&1$ 。

P12 isBetweenEitherOrder:

1. 判断 x 是否介于 a 和 b 之间, 等价于判断 (x-a) 和 (x-b) 是否 “异号或至少有一个为 0”。
2. 为了防止减法溢出, 先检验符号位
若 x 与端点同号, 做减法比较大小;
若 x 与端点异号, 由 x 本身的符号决定大小
3. 得出 $x \geq a$ 和 $x \geq b$ 成立一个 (用异或)
或 x 刚好等于某个端点时返回 1。

P13 mul5Sat:

5 倍分解为 4 倍+1 倍, 如果没有溢出, 则 4x 不能溢出 (即前三位要相同), 且 5 倍不溢出, 按照加法溢出判断, 5x 与 x 要同号
如果正溢出, 则正数越乘越小, 负溢出则负数越乘越大

P14 classifyAdd3:

1. 分两步累加并记录每一步的溢出情况：上溢出记为+1，下溢出记为-1
2. 只有两次溢出的记录和位 0（都不溢出或者相反溢出）才符合条件

P15 floatScaleThreeHalves:

提取符号位 s、阶码 exp。

处理无小数部分或特殊值 ($\text{exp} \geq 150$)：直接返回 uf。

处理 $|x| < 1$ 的情况：

若 $\text{exp} < 126$ （即 $|x| < 0.5$ ）或 $\text{exp} == 126$ 且 $\text{frac} == 0$ ，（即 $|x| == 0.5$ ）：舍入为 0，保留符号位；

其它则舍入为 1.0，保留符号位。

处理 $1 \leq |x| < 2^{23}$ ($127 \leq \text{exp} < 150$) 的情况：

计算小数位数 $\text{shift} = 22 - E$ （其中 $E = \text{exp} - 127$ ）；

构造向偶数舍入偏置量 bias；

uf 加上 bias 后清零低 ($\text{shift} + 1$) 位小数。

P16 floatRoundEven:

1. 提取符号位 s、阶码 exp。

2. 处理无小数部分或特殊值 ($\text{exp} \geq 150$)：直接返回 uf。

3. 处理 $|x| < 1$ 的情况：

若 $\text{exp} < 126$ （即 $|x| < 0.5$ ）或 $\text{exp} == 126$ 且 $\text{frac} == 0$ ，（即 $|x| == 0.5$ ）：舍入为 0，保留符号位；

其它则舍入为 1.0，保留符号位。

4. 处理 $1 \leq |x| < 2^{23}$ ($127 \leq \text{exp} < 150$) 的情况：

计算小数位数 $\text{shift} = 22 - E$ （其中 $E = \text{exp} - 127$ ）；

构造向偶数舍入偏置量 bias；

uf 加上 bias 后清零低 ($\text{shift} + 1$) 位小数。

P17 float_i2f:

先取出符号位，并用无符号运算求绝对值，以正确处理 INT_MIN。

再查找绝对值最高有效位的位置，确定浮点数的阶码。

将有效数对齐到 24 位；若需要丢弃低位，则依据最近偶数规则舍入。

如果舍入导致有效数进位溢出，就增加阶码，最后组合符号位、阶码和尾数。

输入为 0 时直接返回正零。

P18 bitCount:

先构造掩码，将每一位的计数两两相加。

再逐步合并相邻的 2 位、4 位、8 位和 16 位计数，

最终得到整个 32 位整数中 1 的总数。

P19 bitReverse:

依次交换相邻的 1 位、2 位、4 位和 8 位，最后交换高低 16 位。

每轮通过掩码提取，再移位到对应位置并合并

三、对本实验的感受

正数后面几题（10，11，12，13）和浮点数的（16，17）略有难度，整体思路挺复杂的，很多已有的位运算操作（比如安全的比大小之类的）可以推荐个博客或者写点教程来简单引导一下。

思路参考：<https://graphics.stanford.edu/~seander/bithacks.html>